Drug Interaction_rapport

Lise Lefebvre

August 2026

1 Introduction

2 Data

2.1 The OGB-ddi dataset

`ogbl-ddi` is one of the link prediction datasets of the Open Graph Benchmark [4]. It is an undirected and unweighted graph. Each node is a drug, approved or experimental, and each edge means that taking the two drugs together has an effect that is clearly different from what we would expect if they acted separately. The interactions come from DrugBank [10].

The graph has 4,267 nodes and 1,334,889 edges. That is a density of 14.6%: almost one drug pair out of seven is recorded as interacting. This is much denser than the other link prediction datasets of the benchmark, and it will matter for our results. The degrees are also very unequal. Computed on the training edges, the least connected drug has one interaction, the median drug has 446, and the most connected has 2,234, which is more than half of all the drugs in the graph.

The dataset comes with no node features. Every method has to choose its own way of representing the drugs. The standard baselines of the benchmark learn one embedding vector per node during training. We do something else and use PubMedBERT embeddings of the drug descriptions. This is our choice and not part of the benchmark.

Split. The edges are divided 80/10/10 into training, validation and test. The split is not random. OGB uses a *protein-target split*: the edges are separated according to the proteins the drugs bind to in the body. The drugs in the test set therefore mostly target different proteins from those in the training set. The idea is to stop a model from succeeding just because a very similar drug was already in the training data.

Evaluation. The official metric is Hits@20. Each true test edge is ranked against about 100,000 randomly sampled non-interacting pairs, and the metric is the proportion of true edges that end up in the top 20. This is demanding: a pair has to beat almost all of the 100,000 negatives, not just a typical one.

We use AUROC instead. Hits@20 depends on how many negatives you rank against, and our three evaluation sets have very different sizes: 133,117 random negatives against 811 CRESCENDDI pairs. Being in the top 20 out of 811 and in the top 20 out of 133,117 are not the same thing, so the values would not be comparable. AUROC is the probability that a positive edge gets a higher score than a negative one, and it does not depend on the sizes. It is also what most of the works we compare with report on this dataset.

The negatives used in the official evaluation are sampled uniformly among the pairs that are not edges, and nobody checks them. A pair counts as negative because it is absent from the graph. That can mean the two drugs do not interact, or simply that nobody has ever studied the question. What follows from this choice is the subject of Section 4.

		Count	Mean degree
Graph			
Drugs (nodes)		4,267	
Edges before conflict removal		1,334,889	
Edges removed (CRESCENDDI conflicts)	2,233	(0.167%)	
Edges after removal		1,332,656	
Graph density		14.6%	
Positive edges (after removal)			
Training		1,066,276	
Validation		133,263	
Test		133,117	793.8
CRESCENDDI			
Pairs mapped to OGB-ddi		4,049	966.9
of which conflicting with OGB	2,233	(55.1%)	
Training portion		2,834	
Validation portion		404	
Test portion		811	966.9
Negative sets used for evaluation			
Random non-edges		133,117	453.8
Degree-matched non-edges		810	962.2
CRESCENDDI verified		811	966.9
Node features			
PubMedBERT dimension / coverage	768	/ 100%	

Table 1: Main counts of the frozen dataset `frozen_r5`. All degrees are computed on the training edges only, so that no test information enters any reported number. One CRESCENDDI test pair could not be matched by the degree-matched construction, which therefore has 810 pairs instead of 811.

2.2 CRESCENDDI and its relation to OGB-ddi

CRESCENDDI is a set of drug pairs that clinical sources have looked at [6]. It was built to test signal detection methods in pharmacovigilance, not for machine learning, so we have to be careful about what it says.

Its rows are not pairs but triplets: two drugs and one adverse event. There are 10,286 positive controls and 4,544 negative controls, over 454 drugs and 179 events. A triplet is a positive control if the pair appears in all three sources the resource is built from, which are the British National Formulary, the French ANSM Thesaurus and Micromedex, and if the text describes a real clinical effect. Descriptions that only propose a mechanism were left out. A triplet is a negative control if the pair appears in none of the three sources, and if a PubMed query for the two drugs together with the event returns nothing.

So a negative control does not mean that the two drugs do not interact. It means that nobody has published anything about them causing that particular event. The authors say this themselves. It is still much better than what the benchmark does, where a pair is a negative simply because it is not in the graph.

We only need pairs, so we drop the event column. Pairs that appear several times, once per event, become duplicates and we remove them. We then map the drug names to DrugBank identifiers and the identifiers to OGB-ddi nodes. This leaves 4,743 positive pairs out of 10,286 triplets, and 4,049 negative pairs out of 4,544. Five drug names could not be mapped. The two sets do not share a single pair, which is what the construction rules predict.

How many of these pairs are in OGB-ddi?

We then looked at how many of these pairs are edges in OGB-ddi. We cannot compare the result to the density of the graph, which is 14.6%. The drugs in CRESCENDDI are used a lot in the clinic and are therefore very connected, and two connected drugs are more likely to be an edge whatever a compendium says about them. So for each set we also built a null: random pairs drawn from the same degree bins, with no clinical information at all. This is the same idea as the degree-matched control of Section **??**.

Set	n	In OGB-ddi	Null	Difference
CRESCENDDI positives	4,743	97.4%	70.2%	+27.2
CRESCENDDI negatives	4,049	55.1%	59.8%	−4.7

Table 2: Share of CRESCENDDI pairs that are edges in OGB-ddi, next to a null of random pairs with the same degrees. Measured on the original graph, before the edges described below were removed.

The first thing this table tells us is that our mapping works. If we had made a mistake going from drug names to node indices, the positive pairs would not come back as edges 97.4% of the time.

The second thing is more interesting. The two rows do not behave the same way. The positive pairs are edges 27 points more often than their degree alone would give, so OGB-ddi finds almost all the interactions that clinicians have confirmed. The negative pairs are edges 4.7 points less often than their degree would give. The sign is the one we want, but the effect is small. If we treat "being an edge in OGB-ddi" as a test for "is this a real clinical interaction", the test almost never misses a true one, and almost never rejects a false one.

This comes from how the graph was made. OGB-ddi comes from DrugBank [10], which contains interactions predicted from the mechanism of the drugs as well as interactions seen in patients. Being plausible is a much easier condition than being observed. The graph is therefore nearly complete on real interactions, and says very little about pairs that do not interact. The null says the same thing from the other side: between drugs this connected, most pairs are edges anyway.

Removing the conflicting edges

2,233 of the 4,049 negative pairs, that is 55.1%, are edges in OGB-ddi. A pair cannot be a positive edge in the graph and a negative in our evaluation set at the same time, so we have to choose. We remove these 2,233 edges from the graph and keep all 4,049 pairs as negatives. This removes 0.167% of the edges. We do not re-split afterwards, so the edges are taken out of whichever official split they were in.

The drugs involved in these conflicts are not unusual ones. The twenty most frequent are ordinary, widely prescribed medicines (acetaminophen, lorazepam, levothyroxine, lisinopril) with degrees between 1,000 and 2,200, against a mean of about 625 in the graph. Across the 433 drugs appearing in the CRESCENDDI negatives, the share of a drug's pairs that conflict is strongly correlated with its degree (Spearman $\rho = 0.68$). A conflict is therefore mostly a consequence of how much a drug has been studied, rather than of a disagreement about that particular drug.

The 2,233 removed edges fall in the official split as 1,635 training, 226 validation and 372 test. Edges are removed in place rather than triggering a re-split, so each remaining edge keeps its original assignment. The three splits are correspondingly smaller than the official ones: our test set contains 133,117 positive edges instead of 133,489.

We are not removing these edges because we have shown they are wrong. Table 2 shows they appear at roughly the rate the degree alone predicts. We remove them because our evaluation needs the labels to be consistent, and because between the two sources the clinical one is the one we want to be measured against.

Two things come out of Table 3.

First, the degree-matched control works. Its mean degree (962.2) is almost the same as the CRESCENDDI one (966.9), and the medians are identical (925). The two sets differ on one point only: the CRESCENDDI pairs have been verified in clinical compendia, while the matched pairs are ordinary non-

Set	n	Mean degree	Median degree
Random negatives	133,117	453.8	368
Positive edges	133,117	793.8	788
Degree-matched	810	962.2	925
CRESCENDDI	811	966.9	925

Table 3: Mean and median degree of the drugs in each evaluation set.

edges. This is what lets us separate the effect of the degree from the effect of the verification.

Second, and this is the important part, the sets are not in the order one would expect. The random negatives have a low degree (453.8) because most drugs in the graph have few interactions, so a uniform sample mostly picks rarely used drugs. The positive edges are higher (793.8) because real interactions involve drugs that have been well studied. But the CRESCENDDI pairs are higher still (966.9), because they are made of drugs that are used often in the clinic and have therefore been tested against many others.

This has a direct consequence. Take a model that only looks at the degree and predicts an interaction when both drugs are well connected. On random negatives it looks good, because the positives (793.8) are more connected than the negatives (453.8). On CRESCENDDI the same model looks bad, because now the negatives (966.9) are more connected than the positives (793.8), so its ranking is reversed. On CRESCENDDI the degree is not a weak signal, it is a signal that points the wrong way.

The same rule therefore gives a good value on one evaluation set and a value below chance on the other, without the model changing at all. We test this directly in Section 4.

3 Methods

3.1 Experimental setup

The encoder is a 3-layer GCN [5] with hidden dimension 256 and dropout 0.5. The score of a pair is the dot product of the two node representations. The decoder has no parameters of its own, so everything the model learns is in the representations. We train with `BCEWithLogitsLoss` and Adam (lr $= 5 \times 10^{-4}$) for 400 epochs, in full batch, with gradient clipping. Node features are PubMedBERT embeddings of the drug descriptions [2], 768 dimensions, covering all 4,267 drugs.

All results are averaged over 5 seeds. The seed controls the initialisation of the model, the draw of the training negatives, and, where it applies, the subsample of positive edges. The evaluation sets are the same for every seed, so the standard deviations we report describe variability in training, not variability in what is being measured.

We report the value at the last epoch, not at the epoch chosen on validation. Our validation set uses random negatives, and performance there stops improving after about 15 epochs, while performance on the two control sets keeps improving until around epoch 300. Choosing the epoch on validation would therefore stop training long before the control sets have converged. Using a fixed budget also means that no configuration is favoured by a lucky stopping point. Epoch 400 is itself an arbitrary choice; what matters is that it is the same for every run.

3.2 Data leakage

Leakage happens when information from the test set reaches the model during training. The model then looks better than it is. Link prediction is particularly exposed, because the edges are both the input and the labels [11]. Below are the risks in our setup and what we did about each.

1. The encoder seeing the edge it has to predict. In a GCN an edge can be used in two ways: as part of the graph for message passing, or as a label for the decoder. It must not be both. If the encoder aggregates over edge (u, v), then the representations of u and v already contain the fact that they are connected, and asking the decoder to score that pair is not really a prediction. We use only the training edges for message passing. The validation and test edges are scored by the decoder but never added to the graph.

2. Random negatives falling on CRESCENDDI pairs. We sample random negatives uniformly among node pairs, and nothing stops a draw from landing on a pair that is also in CRESCENDDI. If that happens, part of our evaluation set ends up in the training data. We therefore exclude all graph edges and all 4,049 CRESCENDDI pairs when sampling, and we do the same for the degree-matched control. In practice this removes almost nothing, since there are about 9 million possible pairs, but it makes the three evaluation sets exactly independent rather than approximately independent.

3. Using CRESCENDDI both to choose the model and to report the result. At first we used the CRESCENDDI pairs to pick the training epoch and also to report the final value. That is not correct: if we stop at the epoch that gives the best CRESCENDDI result, the CRESCENDDI result we report is too optimistic. We now split the pairs into three disjoint parts (70% train, 10% validation, 20% test). Model selection uses the validation part only, and the test part is scored once, at the end.

4. The same drug appearing in train and test. The 4,049 CRESCENDDI pairs are built from only 430 distinct drugs, so the same drug appears in many pairs. With a split at the level of pairs, 99.2% of the drugs in the test portion also appear in the training portion, each one 15.4 times on average, and 807 of

the 811 test pairs are made of two drugs that were both seen during training. A shortcut of the form "this drug is usually safe" is therefore fully available to the model.

How much it helps is another matter. These 430 drugs have a mean degree of 962: each one interacts with about 962 other drugs while being verified as non-interacting with about 15, a ratio of one to sixty. A model that gave a low score whenever one of these drugs is involved would be right on the verified pairs and wrong on all the others, and would be punished for it on the positive test edges. This limits how much of our results can come from remembering drugs, without ruling it out.

A drug-disjoint split, where no drug appears in more than one part, would answer the question directly. On this dataset it keeps only 51% of the pairs and leaves 39 validation pairs, which is too few to select a model, so we report the numbers above as a limitation instead of running the comparison.

5. Two risks we cannot remove. The first comes from the pretrained language model. PubMedBERT was trained on biomedical abstracts [2], and the DrugBank interactions were curated from that same literature. Some of the edges we ask the model to predict may therefore already be in its input features. This is even more direct for the drug descriptions, which sometimes name the interacting drug explicitly. Embeddings computed from the chemical structure, such as ChemBERTa [1], would not have this problem, since molecular structure is not where interaction reports are written down. We compare the two in Section 4.5. The comparison does not settle the leakage question, because both embeddings turn out to be worse than random vectors for reasons unrelated to their content, so the risk described here remains unmeasured.

The second is the transductive setting. All the drugs are already in the graph when we train, so the model never has to predict for a drug it has not seen [3]. This is the standard protocol for `ogbl-ddi` and it lets us compare with published results, but it limits what our numbers say about new drugs. We come back to this in Section 6.

3.3 The three evaluation sets

Every model in this report is evaluated on the same 133,117 positive test edges, against three different sets of negatives.

- **Random**: 133,117 non-edges sampled uniformly. This is the standard protocol.
- **Degree-matched**: for each CRESCENDDI test pair, a non-edge whose two drugs have the same degrees. Nobody has verified these pairs; they are ordinary non-edges taken from the same region of connectivity.
- **CRESCENDDI**: the 811 verified pairs of the test portion.

Going from the first to the second changes the degree and nothing else. Going from the second to the third changes the clinical verification and nothing else. This is what allows us to say which of the two is responsible for what we observe.

To build the matched set, we bin the nodes by degree on a logarithmic scale, and for each CRESCENDDI test pair we draw a pair of nodes from the same two bins, rejecting any candidate that is an edge, a CRESCENDDI pair, or already drawn. One pair out of 811 could not be matched.

4 Results

4.1 Reproduction of the benchmark

Under the standard protocol, with random negatives for training and for evaluation, our GCN reaches an AUROC of 0.905 ± 0.003 over 5 seeds. For reference, Li et al. [8] report an AUC of 0.900 on the same dataset. The two values are not directly comparable: LLM-DDI uses five-fold cross-validation instead of the official protein-target split, an MLP decoder instead of a dot product, and a preprocessed graph whose statistics differ from the released one. We give their value only to show roughly where ours sits.

What matters for the rest of this section is not that our model is the best one. It is that it is an ordinary one, trained in the usual way and evaluated in the usual way, with a value in the usual range. So what follows cannot be blamed on a weak model.

4.2 A model using only the degree

We now ask how much of that value can be obtained without knowing anything about the drugs. We train a logistic regression with two features: the degrees of the two drugs of the pair. Since the graph is undirected we use the minimum and the maximum of the two, so that the prediction does not depend on the order in which a pair is written, and we take the logarithm of both because the degree distribution is very skewed. The degrees are computed on the training edges only. The model is trained on the same 1,066,276 positive edges and the same number of random negatives as the GCN.

Test negatives	AUROC
Random	0.8799
Degree-matched	0.3280
CRESCENDDI	0.3289

Table 4: Degree-only logistic regression: two features per pair, no chemical and no textual information.

Three things follow from Table 4.

First, this model reaches 0.8799 under the standard protocol. That is 97.2% of the 0.905 obtained by the GCN with 768-dimensional embeddings and three convolution layers. Almost everything the benchmark measures is available from two numbers per pair.

Second, the same model drops to 0.328 and 0.329 on the two other sets. A value below 0.5 does not mean the model has no information; it means its ranking is reversed. The rule it learned, that two well connected drugs are likely to interact, puts the verified negatives above the true positives, because those negatives have a higher mean degree (966.9) than the positives (793.8). The coefficients confirm this: both are positive, +2.42 on the smaller degree of the pair and +0.67 on the larger one. The bigger weight on the smaller degree makes sense given how the graph was built. A pair is only recorded as interacting when both drugs have been studied enough for someone to notice, so it is the less studied of the two that limits the prediction.

Third, the values on the degree-matched set (0.3280) and on CRESCENDDI (0.3289) differ by 0.0009, and the mean scores given to the negatives of the two sets are 0.7929 and 0.7932. This is what should happen: the two sets have the same degrees by construction and the model sees nothing else, so it cannot tell them apart. We take this as a check that the control is correctly built. It also means that any difference between these two sets for another model cannot come from the degree, and must come from the clinical verification.

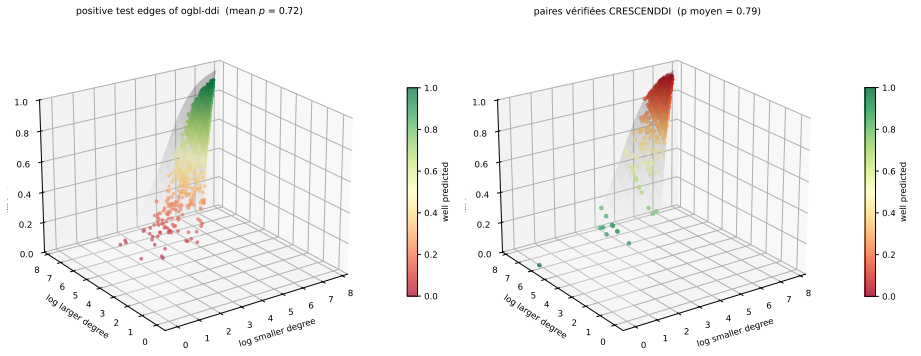


Figure 1: The degree-only model drawn as a surface over the two degrees of a pair, with two evaluation sets on it. Left: the positive test edges, where a high probability is the correct answer. Right: the CRESCENDDI verified negatives, where a low probability is correct. Colour marks correctness in both panels, so green is right and red is wrong, but it is inverted on the right because those pairs are negatives. Points are subsampled.

Figure 1 shows what this model is. It has only two numbers per pair, so its prediction is a single surface over the degree plane, and every point sits exactly on it.

The surface is the same in both panels. Only the region occupied by the

points changes. The positive edges are high on the surface, which is the right answer for them. The CRESCENDDI pairs are higher still, because their drugs are more connected, but they are negatives, so the same answer is now wrong.

It is one rule, applied to two sets that sit in different parts of its input. That is enough to make it right on one and wrong on the other.

4.3 The GCN on the three evaluation sets

Model	Random	Degree-matched	CRESCENDDI
Logistic regression (2 features)	0.8799	0.3280	0.3289
GCN + PubMedBERT (768 dim)	0.905 ± 0.003	0.637 ± 0.009	0.485 ± 0.019

Table 5: AUROC on the three evaluation sets. The GCN values are means and standard deviations over 5 seeds at epoch 400. The logistic regression is deterministic given its training set.

The GCN beats the degree baseline on all three sets, but by very different amounts. On the standard evaluation it gains 0.02. On the two control sets it gains more than 0.15. Whatever the GCN uses beyond the degree is almost invisible in the benchmark number.

Where does the shortcut come from? After a single gradient step, the GCN already gives 0.863, 0.301 and 0.310 on the three sets, which is essentially the profile of the degree model. This comes from message passing itself: the representation of a node is an aggregation over its neighbours, so its norm grows with the number of neighbours, and the dot product of two vectors grows with their norms. Well connected drugs therefore get high scores whatever the weights have learned. The shortcut is there before any learning.

The two quantities also converge at very different speeds. The value on random negatives reaches 0.88 after 25 epochs and barely moves afterwards. The value on the degree-matched set is still 0.36 at epoch 25, 0.58 at epoch 200, and only settles around epoch 300. A training budget chosen on benchmark performance stops long before the model has learned what the control sets measure.

Figure 2 shows what these AUROC values look like. On the standard evaluation the two classes form two separate modes, and the model separates them easily. On the degree-matched set they already overlap heavily. On CRESCENDDI the two distributions lie almost exactly on top of each other.

This is worth stating precisely, because an AUROC of 0.50 can mean two different things. It can mean that the model ranks the classes in the wrong order, or that it does not distinguish them at all. Here it is the second: the model is not giving the verified negatives low scores, it is giving them the same scores as the true interactions. There is no threshold that would separate them.

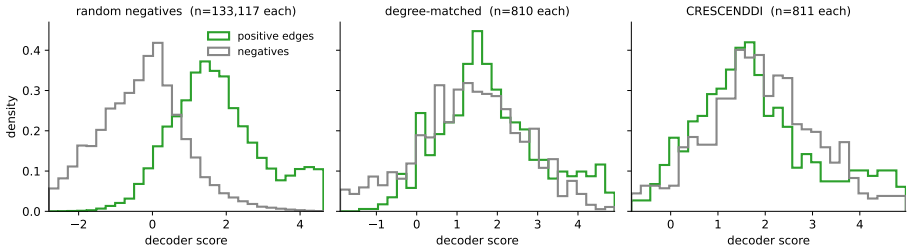


Figure 2: Distribution of the decoder scores for the model trained on random negatives, one panel per evaluation set (seed 0, epoch 400). Raw dot products, before the sigmoid. The positives are subsampled to the size of each negative set, so the two classes have equal counts in every panel. Axes are restricted to the bulk of each distribution.

4.4 Training on verified negatives

Everything above used random negatives during training. We now train the same model on the CRESCENDDI training pairs instead. There are only 2,834 of them against more than one million positive edges, a ratio of about 380 to 1. A loss weight alone cannot handle that, since it cannot create signal out of 2,834 examples, so we first subsample the positives to 10:1 and then reweight the loss so that the two classes contribute equally. The encoder still uses the full graph; only the pairs given to the decoder change.

Training negatives	Random	Degree-matched	CRESCENDDI	Spread
Random	$\mathbf{0.905 \pm 0.003}$	0.637 ± 0.009	0.485 ± 0.019	0.420
CRESCENDDI	0.659 ± 0.005	$\mathbf{0.739 \pm 0.027}$	0.687 ± 0.016	0.080

Table 6: AUROC depending on the negatives used for training and for evaluation. Means and standard deviations over 5 seeds at epoch 400. "Spread" is the difference between the highest and the lowest of the three values.

Training on CRESCENDDI gives a much lower value on the standard evaluation (0.659 against 0.905). That is the price of not using the shortcut. The interesting part is elsewhere: the three values of this model are within 0.080 of each other, against 0.420 for the model trained on random negatives. Its performance depends much less on which negatives are used to evaluate it.

One detail is worth pointing out. The model trained on CRESCENDDI does better on the degree-matched set (0.739) than on the CRESCENDDI set it was actually trained on (0.687). The difference is 0.052 against a standard deviation of 0.027, so it is suggestive rather than established.

The training curves show the shortcut being lost. In the run trained on CRESCENDDI, the value on random negatives falls from 0.863 at epoch 1 to 0.548 at epoch 50, while the value on CRESCENDDI rises over the same interval.

From about epoch 100 the three curves go up together (Figure 3).

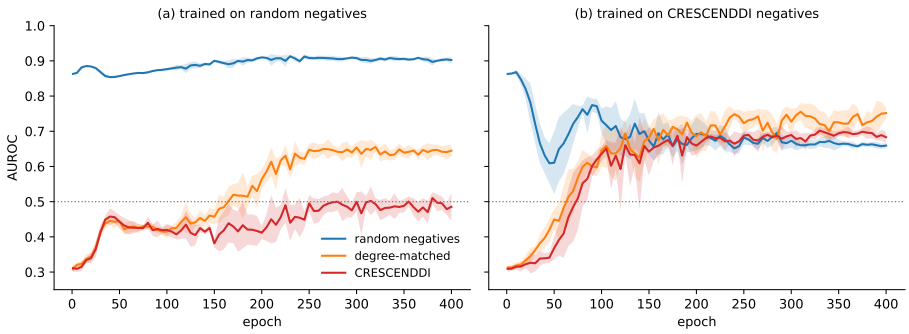


Figure 3: AUROC during training, mean over 5 seeds with a ±1 standard deviation band. The dotted line marks chance. Left: trained on random negatives. Right: trained on CRESCENDDI negatives.

4.5 Do the node features help?

Everything so far used PubMedBERT embeddings as node features. We now ask a simpler question: do these features help at all? We compare several inputs to the same encoder, keeping everything else fixed.

- **Random vectors**: 768 numbers drawn from a standard normal for each drug, the same draw for every run. They contain nothing about the drugs.
- **ChemBERTa** [1]: 768 dimensions computed from the chemical structure of the molecule.
- **PubMedBERT** [2]: 768 dimensions computed from the DrugBank text description. This is the configuration used elsewhere in this report.
- **GPT**: the 1,536-dimensional `text-embedding-ada-002` vectors of the drug descriptions, taken from the DDI-LLM framework [8]. These cover 81.6% of our drugs; the remaining 786 nodes receive the mean vector.
- **The same embeddings after whitening**, described below.

Each is run in both training regimes, on 5 seeds, for 400 epochs.

The embeddings as they come

Take the first four rows of each block. On the standard evaluation they are almost the same, between 0.905 and 0.927. The benchmark cannot tell apart a model that was given the chemistry of the drugs from one that was given 768 random numbers. This is the same conclusion as Section 4, reached a third way.

Features	Random	Degree-matched	CRESCENDDI
Trained on random negatives			
Random vectors	0.927 ± 0.004	0.831 ± 0.003	0.625 ± 0.009
ChemBERTa	0.908 ± 0.005	0.690 ± 0.014	0.536 ± 0.018
GPT (ada-002)	0.905 ± 0.003	0.656 ± 0.021	0.504 ± 0.012
PubMedBERT	0.905 ± 0.003	0.642 ± 0.015	0.490 ± 0.023
ChemBERTa whitened	0.934 ± 0.003	0.860 ± 0.011	0.639 ± 0.008
PubMedBERT whitened	$\mathbf{0.941 \pm 0.004}$	$\mathbf{0.871 \pm 0.014}$	$\mathbf{0.645 \pm 0.014}$
Trained on CRESCENDDI negatives			
Random vectors	0.739 ± 0.009	0.873 ± 0.004	0.795 ± 0.007
ChemBERTa	0.665 ± 0.007	0.777 ± 0.015	0.690 ± 0.012
GPT (ada-002)	0.666 ± 0.005	0.764 ± 0.012	0.680 ± 0.007
PubMedBERT	0.660 ± 0.004	0.744 ± 0.028	0.687 ± 0.017
ChemBERTa whitened	0.756 ± 0.006	0.882 ± 0.006	0.796 ± 0.006
PubMedBERT whitened	$\mathbf{0.757 \pm 0.006}$	$\mathbf{0.889 \pm 0.006}$	$\mathbf{0.795 \pm 0.008}$

Table 7: AUROC for six feature types, means and standard deviations over 5 seeds at epoch 400. The GPT row uses 1,536 dimensions and the others 768, which matters for the comparison; see the discussion of capacity below.

On the two control sets they are not the same at all, and the order is the opposite of what one would expect. The random vectors are the best of the four everywhere, by 0.19 on the degree-matched set in the first regime and by 0.10 in the second. These differences are many times the standard deviation across seeds.

Why the embeddings do worse

Our first idea was that the embeddings might be badly scaled. The loss at the first epoch is about 0.4 for the random vectors, 4 for PubMedBERT and 15 for ChemBERTa, which suggests the model starts far outside a usable range. But the vector lengths do not support this: the mean norm is 27.7 for the random vectors, 26.7 for ChemBERTa and 15.1 for PubMedBERT, so the embeddings are not longer, and PubMedBERT is the shortest of the three.

The explanation is in the angles. Our decoder is a dot product,

$$s_{uv} \;=\; z_u \cdot z_v \;=\; \|z_u\|\,\|z_v\| \cos\theta_{uv},$$

and the two lengths are fixed once the drugs are fixed, so the only thing that varies from pair to pair is $\cos\theta$. For the model to separate interacting from non-interacting pairs, the angles have to vary.

The random vectors point in all directions, as one expects from Gaussian noise in 768 dimensions. The embeddings do not: in PubMedBERT space any two drugs are within about 23° of each other. The vectors have 768 dimensions but sit inside a narrow cone, so there is very little angular variation left for the

Features	Mean norm	Mean $\cos\theta$	Mean angle
Random vectors	27.7	0.000	90°
ChemBERTa	26.7	0.670	48°
GPT (ada-002)	1.0	0.756	41°
PubMedBERT	15.1	0.921	23°

Table 8: Geometry of the feature spaces, over 20,000 random pairs of distinct drugs. The GPT vectors are returned already normalised.

decoder to use. This property is called anisotropy and is known to affect the spaces produced by BERT-like models.

The order of the four spaces, $0.000 < 0.670 < 0.756 < 0.921$, is exactly the order of the four rows in Table 7, reversed, in both training regimes. The less the vectors vary in direction, the worse the model does. The GPT row is a prediction rather than an observation: we measured its cosine before running it, found 0.756, and expected it to land between ChemBERTa and PubMedBERT. It did.

One further detail points the same way. For PubMedBERT the length of a drug's vector is negatively correlated with its degree (Spearman $\rho = -0.22$): the best connected drugs get the shortest vectors, so the features push exactly those pairs towards lower scores, against what the graph structure says. The random vectors show no such correlation ($\rho = 0.01$).

Removing the anisotropy

The argument so far is a correlation between a property of the features and a property of the results. To test it we removed the concentration directly.

The transformation is ZCA whitening. Let X be the matrix of embeddings. We centre it, $X_c = X - \bar{x}$, compute the covariance $\Sigma = \mathrm{cov}(X_c)$ and its eigendecomposition $\Sigma = V\Lambda V^\top$, and set

$$X_w \;=\; X_c\, V\,(\Lambda + \epsilon I)^{-1/2}\, V^\top,$$

with $\epsilon = 10^{-5}$. Reading the factors from left to right, the vectors are rotated into the eigenbasis, each direction is divided by its standard deviation, and they are rotated back. The covariance of X_w is the identity, so no direction is favoured. Rotating back is what makes this ZCA rather than PCA whitening [**?**], and it keeps each whitened vector as close as possible to the one it came from. We then rescale so that the mean norm matches the random vectors, leaving direction as the only difference. The transformation adds no information: it is a fixed linear map on the same numbers, computed without labels. The same idea has been used to correct anisotropy in sentence embeddings [9].

It works. The mean cosine falls from 0.921 to 0.020 for PubMedBERT and from 0.670 to 0.027 for ChemBERTa.

The effect on the results is large. On the degree-matched set with random negatives, PubMedBERT goes from 0.642 to 0.871 and ChemBERTa from

0.690 to 0.860. Both whitened embeddings now beat the random vectors, and whitened PubMedBERT is the best single input in all six columns. The features do carry information about drug interactions; the dot product could not reach it.

Two things did not turn out as expected. We thought the gain would follow the amount of anisotropy removed, since PubMedBERT starts far more concentrated. It does not: PubMedBERT gains 0.229 and ChemBERTa 0.170, but they finish within 0.011 of each other, less than one standard deviation. The anisotropy behaves like a threshold rather than a dial.

We also checked that whitening had not collapsed the two embeddings onto the same representation, which would make their equal performance an artefact. For each drug we took its ten nearest neighbours in each whitened space: the two lists share 0.023 of their members on average. The spaces disagree almost completely about which drugs resemble which, and still give the same AUROC. On this dataset, chemical structure and clinical text are two different and equally usable views of the drugs.

Combining the two embeddings

Since the two whitened spaces encode different things, we tested whether they are complementary, by concatenating them into a 1,536-dimensional input. Doubling the input also doubles the parameters of the first convolution layer, so we ran a 1,536-dimensional random Gaussian input as a control: same capacity, no information.

Features	dim	Degree-matched	CRESCENDDI
Random vectors	768	0.831 ± 0.003	0.625 ± 0.009
PubMedBERT whitened	768	0.871 ± 0.014	0.645 ± 0.014
Random vectors	1,536	0.859 ± 0.011	0.627 ± 0.008
Both whitened, concatenated	1,536	0.886 ± 0.010	0.661 ± 0.010

Table 9: Concatenating the two whitened embeddings, against a random input of the same dimension. Trained on random negatives, 5 seeds.

The control matters. Going from 768 to 1,536 random dimensions is worth +0.028 on its own, with no information added. Against that baseline the concatenation gains +0.027, close to the +0.040 that whitened PubMedBERT gains over the 768-dimensional random vectors. The two embeddings together are therefore not clearly better than the better of the two alone, once capacity is accounted for. This also means the rows of Table 7 are not a single ranking: each should be compared with a random input of its own size, which is why the GPT row cannot be read directly against the others.

What this means

The project began by asking whether biomedical embeddings improve on general-purpose ones. The answer is more specific than the question. Used as they come, with a dot-product decoder, they are worse than random vectors on every evaluation set that is not the benchmark, and the ordering across three different embeddings follows their geometry rather than their content. A linear transformation that adds no information removes the problem, after which both embeddings beat the random vectors and the difference between text and structure disappears.

One consequence reaches beyond our own setting. A study that reports an improvement from semantic features, or the absence of one, may be reporting a property of the embedding geometry rather than of the information the embedding contains. The two are easy to confuse when only one baseline is used.

5 Discussion

Two results in this section point the same way.

A logistic regression with two features reaches 97.2% of the value that a GCN with 768-dimensional biomedical embeddings obtains on the benchmark. And the GCN already produces most of its benchmark value after a single gradient step. Both say that the number reported under the standard protocol is largely a measurement of how connected the drugs are, not of whether they interact.

This is not a property of our model. It follows from two things that are independent of the method: the negatives are sampled uniformly, so they are much less connected than the positives, and message passing makes representations grow with connectivity. Any model combining these two ingredients on a graph with a skewed degree distribution would show the same behaviour.

The 2×2 in Table 6 adds a second point. The configuration with the best benchmark value is also the one whose performance varies most between evaluation sets (spread 0.420), while the configuration with the worst benchmark value is by far the most stable (spread 0.080). Choosing a configuration on its benchmark value therefore selects against robustness on this dataset.

This connects to existing work on link prediction evaluation. Yang et al. [11] already argued that the way negatives are handled makes reported results hard to compare. More recently, Li et al. [7] showed that on `ogbl-ddi` the results reported for several methods do not reproduce, and that validation and test performance are only weakly related under the standard evaluation, which is the same effect as the saturation we observe. Their answer is to sample harder negatives heuristically. Ours is different in one respect: the CRESCENDDI negatives are not harder by construction, they are verified, so a failure on them is a failure on pairs that we know are non-interacting.

Finally, our results do not contradict earlier work on sparser interaction graphs. On BioSnap, whose density is about 4%, the same mapping produces an

overlap with CRESCENDDI of only 4.6% instead of 55.1%, and the gap in degree between positives and random negatives is much smaller. The mechanism we describe needs a dense graph with a skewed degree distribution, which `ogbl-ddi` has and BioSnap does not to the same extent.

6 Limitations

6.1 The method cannot handle a drug it has never seen

Our setting is transductive: all 4,267 drugs are in the graph during training, and the model computes a representation for each of them, including those that only appear in test edges. This is the standard protocol [4], but the restriction is not only one of evaluation.

A GCN builds the representation of a drug by aggregating over its neighbours. A drug that has just been approved has no edges in the interaction graph, so it has no neighbourhood to aggregate over, and the encoder has nothing to compute from. The model is not simply untested on new drugs: it cannot produce a prediction for them at all.

This matters for the application. The drugs where an interaction prediction would be most useful are exactly the recent ones, where the clinical record is thin and an unexpected interaction is most likely to cause harm. A method that needs a drug to be already well connected is least usable where it would be most valuable. The same is true of the degree effect: a model that relies on connectivity gives low scores to poorly studied drugs, which is the right answer about the recorded data and the wrong answer about the clinical question.

Two directions would address this. An inductive architecture such as GraphSAGE [3] aggregates features rather than node identities and can represent a node that was absent at training time, as long as it has some neighbours. A model using only the drug representations, without the graph, would work even for a completely isolated drug, at the cost of the structural information.

6.2 What our results do and do not show

What we show is a property of the evaluation, not a ranking of methods. On this dataset, a large part of what the standard protocol measures is available from the degree alone, is present before training, and does not survive a change in how the negatives are built. That says something about the benchmark. It does not say that graph neural networks are ineffective for interaction prediction, and it does not tell anyone which method to use.

Two consequences. First, our conclusions concern `ogbl-ddi`, and more generally evaluations built on uniformly sampled negatives on dense graphs with skewed degrees. Whether they transfer to sparser graphs is open, and the BioSnap comparison above suggests they may not. Second, we tested one architecture with one decoder. A decoder with its own parameters, or an architecture

that normalises the node representations, might be less exposed to the degree effect than the dot product we use. We did not measure this.

6.3 Remaining risks

Three points from Section 3 are not settled by the controls we put in place.

The features come from a language model pretrained on the same literature the interactions were curated from, so part of what we ask the model to predict may already be in its inputs. We report this as a risk rather than a measured quantity, because measuring it would need a language model trained with the DDI literature removed.

The CRESCENDDI evaluation set has 811 pairs, which is small. The standard deviation across seeds there is 0.019, about seven times the one on the random negatives, and the model sits near 0.5, where AUROC is least stable. The differences we report between training configurations are much larger than that, but the individual values should be read with this width in mind.

Finally, the degree-matched control removes the effect of the degree, but the degree is not the only structural quantity that separates positive edges from uniformly sampled non-edges. The number of common neighbours, the Adamic-Adar index, and belonging to the same dense region of the graph all differ too, and matching on the degree controls them only in part. A control matched on the number of common neighbours as well would be a natural improvement.

7 Conclusion

References

[1] Seyone Chithrananda, Gabriel Grand, and Bharath Ramsundar. Chemberta: Large-scale self-supervised pretraining for molecular property prediction. *arXiv preprint arXiv:2010.09885*, 2020.

[2] Yu Gu, Robert Tinn, Hao Cheng, Michael Lucas, Naoto Usuyama, Xiaodong Liu, Tristan Naumann, Jianfeng Gao, and Hoifung Poon. Domain-specific language model pretraining for biomedical natural language processing. *ACM Transactions on Computing for Healthcare*, 3(1):1–23, 2021.

[3] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[4] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. Open graph benchmark: Datasets for machine learning on graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[5] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

[6] Elpida Kontsioti, Simon Maskell, Bhaskar Dutta, and Munir Pirmohamed. A reference set of clinically relevant adverse drug-drug interactions. *Scientific Data*, 9(1):72, 2022. doi: 10.1038/s41597-022-01159-y.

[7] Juanhui Li, Harry Shomer, Haitao Mao, Shuaiqiang Zeng, Yao Ma, Neil Shah, Jiliang Tang, and Dawei Yin. Evaluating graph neural networks for link prediction: Current pitfalls and new benchmarking. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[8] Xin Li et al. Llm-ddi: Leveraging large language models for drug-drug interaction prediction on biomedical knowledge graph. *IEEE Journal of Biomedical and Health Informatics*, 30(1):778–789, 2026.

[9] Jianlin Su, Jiarun Cao, Weijie Liu, and Yangyiwen Ou. Whitening sentence representations for better semantics and faster retrieval. *arXiv preprint arXiv:2103.15316*, 2021.

[10] David S. Wishart, Yannick D. Feunang, An C. Guo, et al. Drugbank 5.0: a major update to the drugbank database for 2018. *Nucleic Acids Research*, 46(D1):D1074–D1082, 2018.

[11] Yang Yang, Ryan N. Lichtenwalter, and Nitesh V. Chawla. Evaluating link prediction methods. *Knowledge and Information Systems*, 45(3):751–782, 2015. doi: 10.1007/s10115-014-0789-0.